

Fisher-Kolmogorov

Equation for neurodegenerative diseases

Lorenzo Esposito 10765981, Francesco Grassi 10841139,
Giorgio Venezia 10807335

September 18, 2025

Version 1.1.0

Contents

1	Introduction	1
2	Problem model definition	3
2.1	Kinetics of prion-like spreading	3
2.1.1	Diffusion tensor	4
2.2	Weak formulation	5
2.3	Time discretization	5
2.4	Linearization	6
2.5	Finite element setting and final assembly	7
3	Set up and computation	9
3.1	1D and 3D: Key aspects	9
3.1.1	The mesh	9
3.1.2	Diffusion Term	9
3.1.3	Similarity	10
3.1.4	Differences	10
3.1.5	Parameters	10
3.2	Implementation	11
3.2.1	Implementation in 1D	11
3.2.2	Implementation in 3D	12
	Parallelization	12
	Parameters representation	12
4	Performance evaluation	13
4.1	Performance Analysis	13
4.1.1	Setup Time	13
4.1.2	Solve Time	14
4.1.3	Total Time	15
4.2	Different parameters	15
4.3	Stability and Convergence	16
4.3.1	Lax-Milgram Analysis for the Linearized Step	17
	Assumptions	17
	Coercivity of the Bilinear Form $a(\delta, v)$	17
4.3.2	Continuity of the Bilinear Form $a(u, v)$	18
4.3.3	Continuity of the Linear Functional $F(v)$	19
	Summary of Conditions	20

CONTENTS

5 Conclusion	23
Bibliography	25

1 Introduction

Neurodegenerative diseases such as Alzheimer's, Parkinson's, and ALS involve progressive neuronal death caused by misfolded proteins that spread through brain tissue. These diseases follow a prion-like mechanism in which pathogenic proteins convert normal proteins into toxic aggregates, creating cascading propagation across neural networks.

Mathematical modeling uses reaction-diffusion equations to capture this spreading process. The models treat protein aggregation as a non-linear reaction coupled with anisotropic diffusion through brain tissue, reducing the biology mechanism to equations governing protein concentration dynamics in space and time.

This project implements a physics-based reaction-diffusion model combining the Fisher-Kolmogorov equation with directional diffusion terms inspired by the framework presented in the article [1]. The model simulates growth and diffusion in terms of concentration of misfolded protein.

If these models are accurate, they can predict disease progression patterns, identify vulnerable brain regions, and inform therapeutic strategies targeting early-stage protein aggregation before irreversible neuronal damage occurs.

2 Problem model definition

The mathematical formulation of the problem is given by

$$\begin{cases} \frac{\partial c}{\partial t} - \nabla \cdot (D \cdot \nabla c) - \alpha c(1 - c) = 0 & \text{in } \Omega \times [0, T], \\ D \cdot \nabla c \cdot \vec{n} = 0 & \text{in } \partial\Omega \times [0, T], \\ c(t = 0) = c_0 & \text{in } \Omega. \end{cases} \quad (2.1)$$

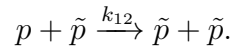
where $c : [0, T] \times \Omega \rightarrow \mathbb{R}$ is the unknown function, $D \in \mathbb{R}^{n \times n}$ is the diffusion tensor, $\alpha \in \mathbb{R}^+$ is a given parameter, $\Omega \subset \mathbb{R}^n$ is a bounded domain, and $\vec{n}$ denotes the outward unit normal vector on $\partial\Omega$.

2.1 Kinetics of prion-like spreading

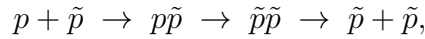
To describe the fundamental mechanism behind prion-like spreading, the article [1] presents a simplified kinetic model, which distinguishes between two states of a protein: the natural healthy form p and the misfolded, pathogenic form $\tilde{p}$. The misfolded proteins propagate through the brain by converting healthy proteins into their misfolded form, in an autocatalytic fashion.

The model considers the following kinetic processes:

- **Recruitment and conversion:** Healthy proteins bind to misfolded proteins and adopt the misfolded state at a rate k_{12} :



More explicitly, this overall reaction is the result of a chain of steps:



where the first step corresponds to recruitment, the second to conversion, and the last to fragmentation of the misfolded dimer.

- **Fragmentation:** Misfolded proteins can fragment to generate new infectious seeds (although this process is not explicitly retained in the final formulation).
- **Production and clearance:** Healthy proteins are continuously produced at a constant rate k_0 and cleared at rate k_1 , while misfolded proteins are cleared at a separate rate $\tilde{k}_1$.

- **Diffusion:** Both types of proteins diffuse in the surrounding medium, with diffusion tensors D_p and $D_{\tilde{p}}$.

The model leads to a system of coupled PDEs governing the evolution of the concentrations of the two protein types:

$$\frac{\partial p}{\partial t} = \operatorname{div}(D_p \nabla p) + k_0 - k_1 p - k_{12} p \tilde{p}, \quad (2.2)$$

$$\frac{\partial \tilde{p}}{\partial t} = \operatorname{div}(D_{\tilde{p}} \nabla \tilde{p}) - \tilde{k}_1 \tilde{p} + k_{12} p \tilde{p}. \quad (2.3)$$

To simplify the system, [1] assumes that the concentration of healthy protein is significantly higher than that of the misfolded form (i.e., $p \gg \tilde{p}$), allowing a quasi-steady-state approximation for p . Under this assumption, the healthy protein concentration is approximated as

$$p \approx \frac{k_0}{k_1 + k_{12} \tilde{p}}. \quad (2.4)$$

Expanding this expression in a Taylor series around $\tilde{p} = 0$ and substituting it into the equation for $\tilde{p}$ yields a nonlinear reaction–diffusion equation.

To further simplify the model, a normalized concentration is introduced for misfolded proteins:

$$c = \frac{\tilde{p}}{\tilde{p}_{\max}}, \quad 0 \leq c \leq 1. \quad (2.5)$$

This leads to a logistic-type equation:

$$\frac{\partial c}{\partial t} = \operatorname{div}(D \nabla c) + \alpha c(1 - c), \quad (2.6)$$

where $D = D_{\tilde{p}}$ is the diffusion tensor and α is the effective growth rate defined as

$$\alpha = \frac{k_{12} k_0}{k_1} - \tilde{k}_1. \quad (2.7)$$

Equation 2.7 captures the key dynamics of prion-like propagation: the diffusion of misfolded proteins and their growth due to the conversion of healthy proteins. The growth rate α increases with the conversion rate k_{12} and the equilibrium healthy protein concentration $p_0 = k_0/k_1$, and decreases with the clearance rate of misfolded proteins $\tilde{k}_1$.

2.1.1 Diffusion tensor

If $\Omega \subset \mathbb{R}^3$ the diffusion term is represented by a tensor $\mathbf{D}$, which accounts for both isotropic and anisotropic transport of misfolded proteins:

$$\mathbf{D} = d^{\text{ext}} \mathbf{I} + d^{\text{axn}} \mathbf{n} \otimes \mathbf{n}. \quad (2.8)$$

Here, d^{ext} denotes the isotropic extracellular diffusion, while d^{axn} represents anisotropic transport along axonal fibers. The unit vector $\mathbf{n}$ indicates the local fiber direction.

Else, if $\Omega \subset \mathbb{R}$, the diffusion term reduces to a scalar coefficient:

$$D = d \in \mathbb{R}, \quad (2.9)$$

since anisotropic effects cannot be represented in a single spatial dimension.

2.2 Weak formulation

As for most partial differential equations, the first step is to construct the weak formulation.

1. To derive the weak formulation, we introduce the Hilbert space

$$H^1(\Omega) = \left\{ v \in L^2(\Omega) : \nabla v \in [L^2(\Omega)]^d \right\}.$$

It is required that $c \in H^1(\Omega)$ in order to be a solution of Equation 2.1.

2. For a function $c \in H^1(\Omega)$ to be a solution of Equation 2.1, it is required that $\forall v \in H^1(\Omega)$:

$$\int_{\Omega} \frac{\partial c}{\partial t} v \, d\mathbf{x} - \int_{\Omega} \nabla \cdot (D \nabla c) v \, d\mathbf{x} - \int_{\Omega} \alpha c(1 - c) v \, d\mathbf{x} = 0. \quad (2.10)$$

3. By integrating by parts the second term, $-\int_{\Omega} \nabla \cdot (D \nabla c) v \, d\mathbf{x}$, we obtain:

$$-\int_{\Omega} \nabla \cdot (D \nabla c) v \, d\mathbf{x} = \int_{\Omega} D \nabla c \cdot \nabla v \, d\mathbf{x} - \int_{\partial\Omega} (D \nabla c \cdot \vec{n}) v \, ds. \quad (2.11)$$

The boundary term $\int_{\partial\Omega} (D \nabla c \cdot \vec{n}) v \, ds$ vanishes due to the second condition in Equation 2.1. Substituting into Equation 2.10, we obtain the weak formulation of Equation 2.1:

$$\int_{\Omega} \frac{\partial c}{\partial t} v \, d\mathbf{x} + \int_{\Omega} D \nabla c \cdot \nabla v \, d\mathbf{x} - \int_{\Omega} \alpha c(1 - c) v \, d\mathbf{x} = 0. \quad (2.12)$$

2.3 Time discretization

Since Equation 2.12 contains the term $\int_{\Omega} \frac{\partial c}{\partial t} v \, d\mathbf{x}$, it is necessary to discretize the time derivative. We choose the Backward Euler method (BEM) for two reasons. First, it is unconditionally stable, since it can be regarded as a special case of the θ -method with $\theta = 1$, where unconditional stability holds for $\theta \geq 1/2$. In contrast, for example, the Forward Euler method is not unconditionally stable. The second reason is that BEM is less computationally expensive than other unconditionally stable methods, such as the Midpoint method.

We consider the time interval $(0, T]$, where T is the final simulation time, and divide it into N equal subintervals of length $\Delta t = T/N$. For each time step t^{n+1} , the time derivative can be approximated as

$$\left. \frac{\partial c}{\partial t} \right|_{t^{n+1}} \approx \frac{c^{n+1} - c^n}{\Delta t},$$

We define the trial and test space as $V = H^1(\Omega)$ which leads to the fully discrete weak formulation:

Find $c^{n+1} \in V$ such that for all $v \in V$ it holds that

$$\int_{\Omega} \frac{c^{n+1} - c^n}{\Delta t} v \, d\mathbf{x} + \int_{\Omega} D \nabla c^{n+1} \cdot \nabla v \, d\mathbf{x} - \int_{\Omega} \alpha c^{n+1}(1 - c^{n+1}) v \, d\mathbf{x} = 0. \quad (2.13)$$

2.4 Linearization

It is still not possible to utilize Equation 2.13 for the numerical solution because it contains a nonlinear term, namely

$$- \int_{\Omega} \alpha c^{n+1} (1 - c^{n+1}) v \, dx.$$

To solve the problem, we need to use a linearized form of Equation 2.13. This can be done using the Fréchet derivative.

For a given functional $G : V \rightarrow \mathbb{R}$, the Fréchet derivative at a point $u \in H^1(\Omega)$ is a linear operator $A(u) = \frac{dG}{du} : V \rightarrow \mathbb{R}$ such that

$$\lim_{\|d\| \rightarrow 0} \frac{|G(u+d) - G(u) - A(u)d|}{\|d\|} = 0.$$

We now define the nonlinear residual:

$$\begin{aligned} R(c^{n+1} + \delta, v) &= \int_{\Omega} \frac{c^{n+1} + \delta - c^n}{\Delta t} v \, dx + \int_{\Omega} D \nabla (c^{n+1} + \delta) \cdot \nabla v \, dx \\ &\quad - \int_{\Omega} \alpha (c^{n+1} + \delta) (1 - c^{n+1} - \delta) v \, dx. \end{aligned} \quad (2.14)$$

Subtracting $R(c^{n+1}, v)$, we obtain:

$$\begin{aligned} R(c^{n+1} + \delta, v) - R(c^{n+1}, v) &= \int_{\Omega} \frac{\delta}{\Delta t} v \, dx + \int_{\Omega} D \nabla \delta \cdot \nabla v \, dx \\ &\quad - \int_{\Omega} \alpha (\delta (1 - 2c^{n+1}) - \delta^2) v \, dx. \end{aligned} \quad (2.15)$$

Approximating δ^2 as a higher-order small term, we arrive at:

$$\begin{aligned} R(c^{n+1} + \delta, v) - R(c^{n+1}, v) &= \int_{\Omega} \frac{\delta}{\Delta t} v \, dx + \int_{\Omega} D \nabla \delta \cdot \nabla v \, dx \\ &\quad - \int_{\Omega} \alpha \delta (1 - 2c^{n+1}) v \, dx + \mathcal{O}(\delta^2). \end{aligned} \quad (2.16)$$

The right-hand side of Equation 2.16, up to $\mathcal{O}(\delta^2)$, corresponds to the action of the Fréchet derivative:

$$\frac{dR}{dc}(c^{n+1})(\delta, v) = a(c^{n+1})(\delta, v).$$

Thus, solving Equation 2.13 is equivalent to finding $c^{n+1} \in H^1(\Omega)$ such that

$$R(c^{n+1}, v) = 0, \quad \forall v \in H^1(\Omega).$$

To this end, we employ Newton's method

Given an initial guess $c^{(0)}$,

while $k = 1, 2, \dots, N$ **and not converged do**

find $\delta^{(k)}$ such that

$$a(c_k^{n+1})(\delta^{(k)}, v) = -R(c_k^{n+1}, v), \quad \forall v \in H^1(\Omega),$$

and update

$$c^{(k+1)} = c_k^{n+1} + \delta^{(k)}.$$

end while

The problem

$$a(c_k^{n+1})(\delta^{(k)}, v) = -R(c_k^{n+1}, v)$$

is a linear variational problem, which can be solved using the finite element method.

2.5 Finite element setting and final assembly

After deriving a linear differential form of the original problem in Equation 2.1 in the form

$$a(c_k^{n+1})(\delta^{(k)}, v) = -R(c_k^{n+1}, v),$$

we discretize the problem in space using the Galerkin finite element method. To this end, we consider a finite-dimensional vector space $V_h \subset H^1(\Omega)$ with dimension N_h . Our goal is to find $\delta_h^{(k)}$ such that

$$a(c_h^{(k)})(\delta^{(k)}, v_h) = -R(c_h^{(k)}, v_h) \quad \forall v_h \in V_h.$$

We introduce the basis of V_h , denoted by $\{\phi_j\}_{j=1}^{N_h}$, and expand the unknowns as

$$c_h^{(k)}(\mathbf{x}) = \sum_{j=1}^{N_h} c_j^{(k)} \phi_j(\mathbf{x}), \quad \delta^{(k)}(\mathbf{x}) = \sum_{j=1}^{N_h} \delta_j^{(k)} \phi_j(\mathbf{x}).$$

Substituting these expressions into the weak problem and choosing the test functions as $v_h = \phi_i$ gives

$$a\left(\sum_{j=1}^{N_h} c_j^{(k)} \phi_j(\mathbf{x})\right)\left(\sum_{j=1}^{N_h} \delta_j^{(k)} \phi_j, \phi_i\right) = -R\left(\sum_{j=1}^{N_h} c_j^{(k)} \phi_j(\mathbf{x}), \phi_i\right). \quad (2.17)$$

After straightforward computation, the following linear system is obtained:

$$\left(\frac{M}{\Delta t} + K - J(c_k^{n+1})\right) \delta^{(k)} = -M \frac{c_k^{n+1} - c^n}{\Delta t} - K c_k^{n+1} + N(c_k^{n+1}), \quad (2.18)$$

where:

- $\delta^{(k)}$ is the vector of unknown coefficients at Newton iteration k ;
- c_k^{n+1} is the current approximation of the concentration at time step $n+1$;
- c^n is the concentration computed at the previous time step n ;
- $M = (m_{ij})$, $m_{ij} = \int_{\Omega} \phi_i \phi_j d\mathbf{x}$ is the mass matrix;

- $K = (k_{ij})$, $k_{ij} = \int_{\Omega} D \nabla \phi_i \cdot \nabla \phi_j \, d\mathbf{x}$ is the stiffness matrix;
- $J(c_k^{n+1}) = (j_{ij})$, $j_{ij} = \int_{\Omega} \alpha (1 - 2c_k^{n+1}) \phi_i \phi_j \, d\mathbf{x}$ is the Jacobian of the reaction term;
- $N(c_k^{n+1}) = (n_i)$, $n_i = \int_{\Omega} \alpha c_k^{n+1} (1 - c_k^{n+1}) \phi_i \, d\mathbf{x}$ is the nonlinear reaction vector.

For clarity, we introduce the shorthand notation

$$LHS = \frac{M}{\Delta t} + K - J(c_k^{n+1}), \quad RHS = -M \frac{c_k^{n+1} - c^n}{\Delta t} - K c_k^{n+1} + N(c_k^{n+1}).$$

At each time step $n+1$, the Newton method is applied to solve for $\delta^{(k)}$ by solving the linear system in Equation 2.18. The increment $\delta^{(k)}$ is then used to update the solution. At every iteration of the Newton method, both the matrix LHS and the vector RHS are recomputed using the current approximation of the solution c_k^{n+1} .

3 Set up and computation

The problem has been solved using C++ combining `deal.ii` library and MPI for the purpose of parallelization.

3.1 1D and 3D: Key aspects

The problem was initially solved in 1D as a proof of concept and to check feasibility, then scaled to 3D.

3.1.1 The mesh

The mesh used for 3D computation is read from an external `.msh` file provided by the original author of the article [1]. The implementation distinguishes between different brain tissue types through material IDs, with the code specifically tracking Grey Matter (material ID 1) and White Matter(material ID 2). A material mapping system associates each cell with its corresponding tissue type, enabling tissue-specific parameter assignment.

In 1D, the mesh is simply an interval $[0, l] \in \mathbb{R}$ divided into N equal subintervals. For our computation, we have chosen $l = 2$ and $N = 200$.

3.1.2 Diffusion Term

As mentioned in subsection 2.1.1, the diffusion term differs between the 1D and 3D formulations.

In the 3D formulation, the diffusion term is represented by a tensor $\mathbf{D}$ that depends on the fiber direction $\mathbf{n}$. The fiber direction can be defined according to different anatomical orientation models:

- **Circumferential model:** fibers follow a path tangent to the cortical layers, forming circumferential rings.
- **Radial model:** fibers are oriented radially from the center of the brain outward toward the cortical surface.
- **Axonal model:** fibers are oriented along the principal direction of neural axons. In this case, diffusion in white matter follows the circumferential model, while in gray matter it follows the radial model, as shown in [1].

These models allow the diffusion tensor to capture anisotropic spreading of misfolded proteins along specific tissue structures.

In the 1D simplification, the diffusion term reduces to a scalar coefficient:

$$D = d \in \mathbb{R},$$

since anisotropic effects cannot be represented in a single spatial dimension.

3.1.3 Similarity

The methods used to solve equation had to be kept unchanged, indeed the core structure of the algorithm stays unaltered:

- Newton’s method to solve nonlinear systems.
- Time-stepping stucture with backward Euler discretization.
- GMRES with preconditioner as linear solver.

3.1.4 Differences

- 1D implementation uses a less complex and uniform mesh built with

`subdivided_hyper_cube()`.

while 3D implementation reads the mesh directly from the proper file.

- In 1D the diffusion coefficient is just a scalar:

```
const double diffusion_coefficient_loc = diffusion_coefficient.value();
```

while in 3D we model an anisotropic diffusion with tensor formulation:

```
const Tensor<2, dim> D = d_ext_matrix + d_axn_loc * normal_matrix;
```

- In 1D we assume uniform material properties throughout the domain, while the 3D program handles multiple brain tissue types (grey matter, white matter) through a gradient of α .

3.1.5 Parameters

The choice of the parameters is crucial for convergence, especially for 3D simulations. In this study, the inputs and parameters of the problem are:

1. T : final time of the simulation.
2. Δt : time step used for time discretization.
3. Mesh: a 3D discretized representation of the domain in $\mathbb{R}^3$.

4. Degree: the polynomial degree of the finite elements used in the finite element method.
5. d^{ext} : isotropic extracellular diffusion coefficient.
6. d^{axn} : anisotropic axonal transport coefficient along fiber directions.
7. Diffusion type: defines the orientation of misfolded protein transport. It can be:
 - Axonal
 - Circumferential
 - Radial

These types of diffusion are also considered in [1].

8. α : growth rate of misfolded protein concentration.
9. Initial seed: initial non-zero concentration.
10. Seed region: subdomain in which the initial concentration is equal to the initial seed. In 3D simulations, a point x_0 is provided, and the program constructs a spherical region centered at x_0 .

3.2 Implementation

3.2.1 Implementation in 1D

To verify the correctness of our approach, we implemented a 1D solver following the methodology described in [1] and compared the results. In this case, the computational domain is the interval $[0, 2] \subset \mathbb{R}$. The diffusion term D reduces to a scalar parameter d , and the initial seed region consists of a single point at the center of the domain ($x = 0$).

The mesh is created over the interval with $N = 200$ elements and polynomial degree equal to 1. After initializing all necessary objects, as done in the laboratory sessions, the method `solve()` executes the time-stepping procedure, computing the solution at each time step and writing the output to a `.vtu` file.

The method `solve()` internally calls `solve_newton()`, which performs Newton iterations to solve the nonlinear system. For each iteration, the system is linearized around the current solution by invoking `assemble_system()`, and the resulting linear system is solved using `solve_linear_system()`. This process repeats until the norm of the residual vector falls below a fixed threshold, set to 10^{-6} in our code.

`assemble_system()` follows the same structure used in the laboratory exercises to compute the *LHS* matrix and the *RHS* vector. These terms represent the linearized versions of the original system, obtained using the values of the current and previous solutions along with the relevant parameters.

Since homogeneous Neumann boundary conditions are applied and Dirichlet conditions are absent, there is no need to handle boundary conditions explicitly.

`solve_linear_system()` assumes that the *LHS* matrix and *RHS* vector are correctly assembled. We use the Generalized Minimal Residual Method (GMRES) to solve the resulting linear system, and the SSOR preconditioner is applied to accelerate convergence.

3.2.2 Implementation in 3D

The primary differences in the 3D solver include the use of parallelization, the application of different parameters (as described in subsection 3.1.5), and the presence of two distinct material regions.

Parallelization

The 3D solver involves a substantially higher computational cost due to the large number of nodes in 3D meshes and the nonlinear nature of the problem. To make the computation feasible, parallelization is employed using Trilinos with MPI. The mesh is partitioned across multiple processes, with each process handling its assigned degrees of freedom and communicating with neighboring partitions at the boundaries. This domain decomposition allows the workload to be distributed efficiently across processes.

Parameters representation

- **Mesh:** as described in subsection 3.1.5, the mesh is imported from an external `.msh` file. Each element in the file includes a material ID, which can be 1 for Grey Matter or 2 for White Matter.
- **Piecewise function:** following [1], the parameters d^{axn} , d^{ext} , and α are assumed to be piecewise constant with respect to the material ID. This is implemented using a `std::map<unsigned int, double>`, where each entry represents the value for a specific material ID (first entry for Grey Matter, second for White Matter). In the solver, the function `virtual double value()` takes an additional argument `const unsigned int component` to select the material ID corresponding to the point being evaluated. The `component` value must be 1 or 2 to distinguish between Grey and White Matter.
- **Initial seed:** the initial seed region is computed as the intersection of a spherical region centered at the given seed point with the computational mesh.

In the 3D implementation, the SSOR preconditioner used in 1D was replaced by an ILU preconditioner due to better convergence behavior in the higher-dimensional problem.

4 Performance evaluation

The only case worth evaluating is the 3D implementation, as the 1D solver serves only as a proof of concept. In this section, we analyze the performance of the 3D solver in terms of scalability.

The results presented here are obtained using the parameter set from the first case in [1]:

- $T = 20$ year
- $\Delta t = 0.1$ year
- $d^{\text{ext}} = [1.5 \text{ cm}^2/\text{year}, 1.5 \text{ cm}^2/\text{year}]$
- $d^{\text{axn}} = [0.0 \text{ cm}^2/\text{year}, 3.0 \text{ cm}^2/\text{year}]$
- $\alpha = [0.3/\text{year}, 0.6/\text{year}]$
- Diffusion type: Radial

In the vectors above, the first component corresponds to White Matter, while the second component corresponds to Grey Matter.

Mesh Information	
Number of elements	501,458
Finite Element Space	
Degree	1
DoFs per cell	4
Quadrature points per cell	6
DoF Handler	
Number of DoFs	107,394

Table 4.1: Mesh and finite element space statistics.

Table 4.1 summarizes the characteristics of the mesh and the finite element space used in the 3D simulations.

4.1 Performance Analysis

4.1.1 Setup Time

Figure 4.1 shows the setup time of the program with respect to the number of processes used. The setup time decreases until four processes are employed, after

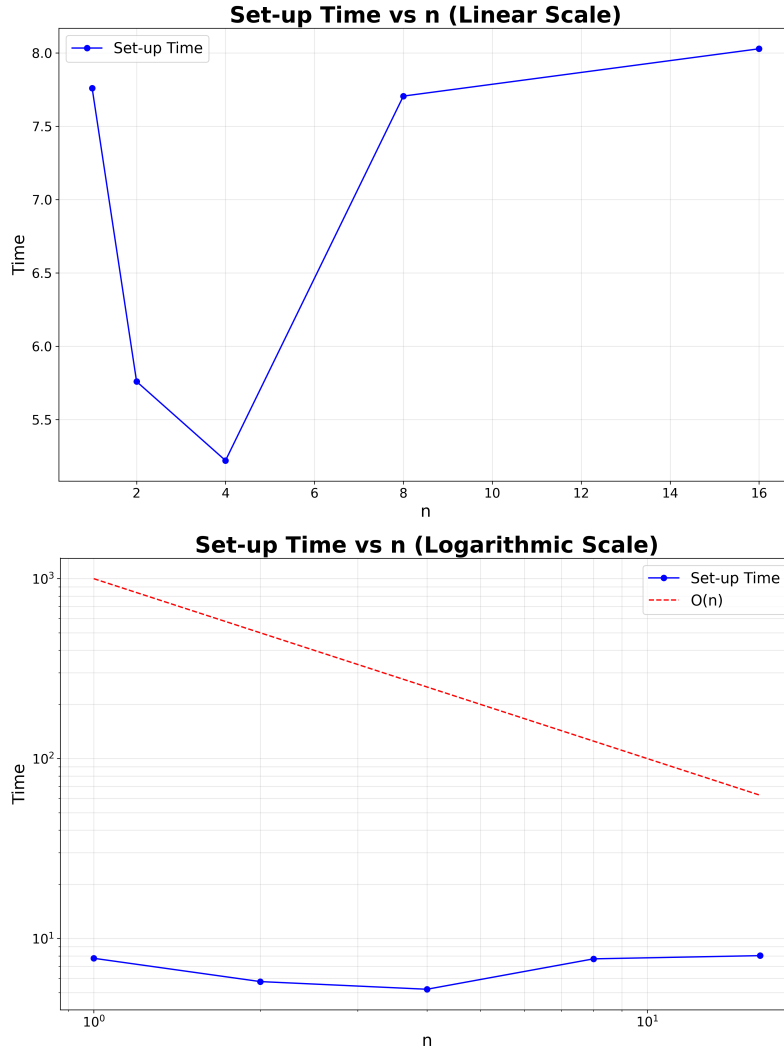


Figure 4.1: Setup time of the program as a function of the number of processes. The top figure is linear scale, the bottom figure is log-log scale.

which it starts to increase. This behavior is caused by the communication overhead inherent in the setup phase; as the number of processes increases, the cost of inter-process communication outweighs the benefits of parallelization.

4.1.2 Solve Time

Figure 4.2 shows the time required to solve the problem as a function of the number of processes. The solve time decreases as the number of processes increases, but the reduction is not linear. This is due to the fact that parts of the algorithm, such as the assembly of the system matrices and the Newton iterations, are inherently sequential or require synchronization between processes. As a result, the efficiency of parallelization diminishes beyond four processes, and the solve time begins to saturate.

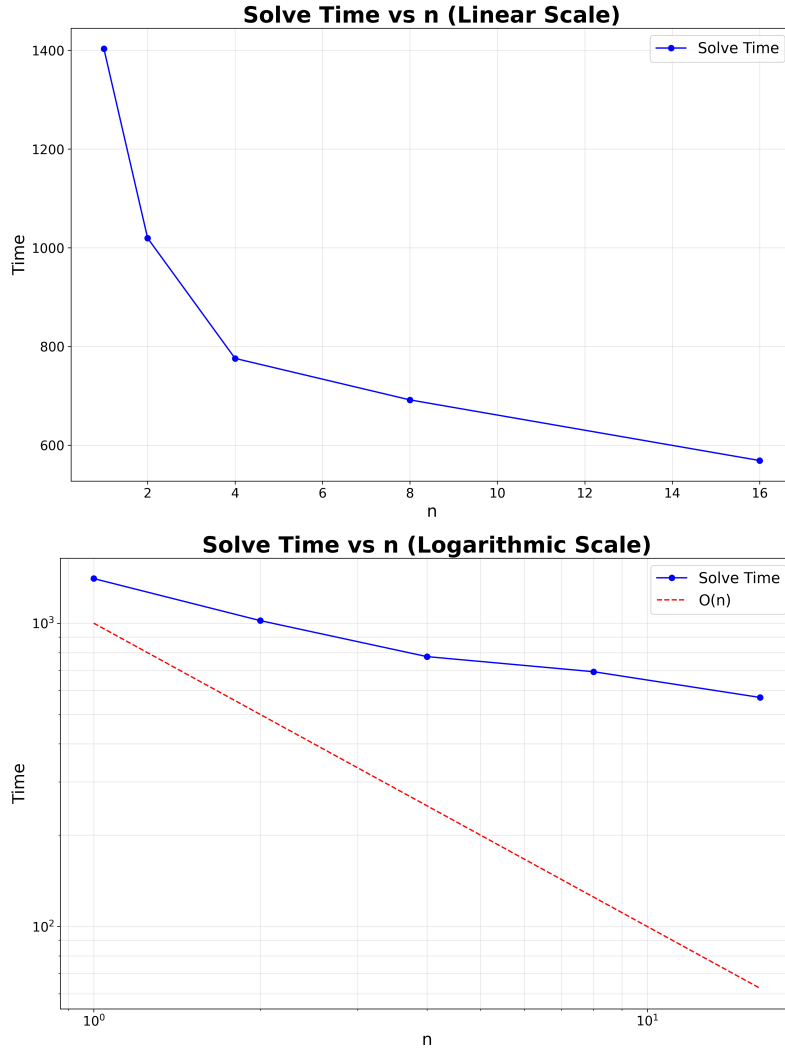


Figure 4.2: Solve time of the program as a function of the number of processes. The top figure is linear scale, the bottom figure is log–log scale.

4.1.3 Total Time

Figure 4.3 reports the total runtime, which includes both setup and solve phases. As expected, the total time decreases with an increasing number of processes, but the improvement is not linear. Beyond four processes, the benefit of adding more processes is minimal.

4.2 Different parameters

We also tested the program with different parameters, in particular we used the second case of parameters from the paper, so with:

- $T = 20$ years
- $\Delta t = 1/6$ years

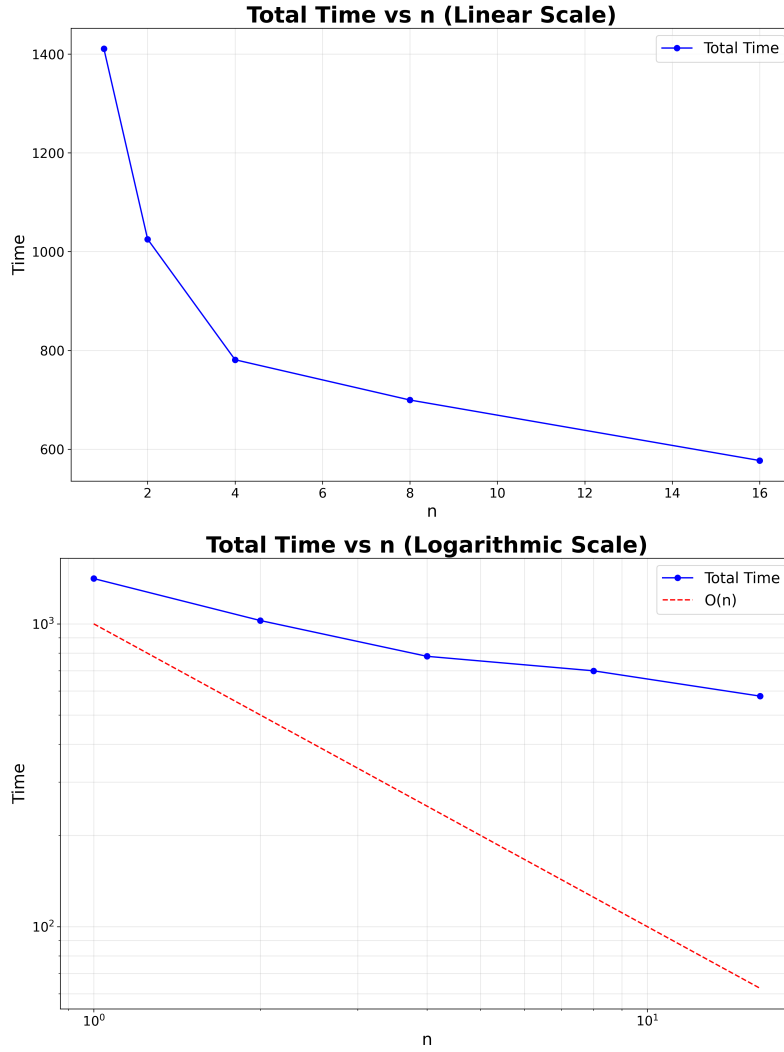


Figure 4.3: Total runtime of the program as a function of the number of processes. The top figure is linear scale, the bottom figure is log–log scale.

- $d^{ext} = [1.5, 1.5]cm^2/year$
- $d^{axn} = [0.0, 3.0]cm^2/year$
- $\alpha = [0.3, 0.6]/year$

Where, again, the first component of the vectors refers to White Matter and the second to Grey Matter.

In both the two configurations, the program was able to run without any issues, and the results were consistent with the expected behavior of the model, compared to the results presented in the paper [1].

4.3 Stability and Convergence

We will not analyze the stability and convergence of the whole project. However, to ensure that each step of our numerical method is well-posed, we can analyze the

linearized problem solved within each Newton iteration. We will demonstrate that this linear problem has a unique solution by verifying the conditions of the Lax-Milgram theorem. This reduces the problem at each iteration to a standard elliptic problem, a topic covered in the course. Finally we also must verify the continuity of both the bilinear form $a(u, v)$ and the linear functional $F(v)$.

4.3.1 Lax-Milgram Analysis for the Linearized Step

At each time step $n + 1$ and for each Newton iteration k , we solve for an update $\delta^{(k)} \in H^1(\Omega)$ that satisfies the linear weak formulation:

$$a(\delta^{(k)}, v) = F(v) \quad \forall v \in H^1(\Omega).$$

This corresponds to the system $a(c_k^{n+1})(\delta^{(k)}, v) = -R(c_k^{n+1}, v)$. To maintain clarity in this section, we relabel the unknown update $\delta^{(k)}$ as δ . The bilinear form $a(\cdot, \cdot)$ and linear functional $F(\cdot)$ are defined as:

$$\begin{aligned} a(\delta, v) &= \int_{\Omega} \frac{1}{\Delta t} \delta v \, d\mathbf{x} + \int_{\Omega} D \nabla \delta, \nabla v \, d\mathbf{x} - \int_{\Omega} \alpha (1 - 2c_k^{n+1}) \delta v \, d\mathbf{x}, \\ F(v) &= - \int_{\Omega} \frac{c_k^{(n+1)} - c^n}{\Delta t} v \cdot d\mathbf{x} - \int_{\Omega} D \nabla c_k^{(n+1)}, \nabla v \, d\mathbf{x} + \int_{\Omega} \alpha c_k^{(n+1)} (1 - c_k^{(n+1)}) v \, d\mathbf{x}. \end{aligned}$$

Assumptions

The analysis relies on the following reasonable assumptions:

- **Function Space:** The solution and test functions belong to $V = H^1(\Omega)$, with the norm $\|v\|_{H^1}^2 = \|v\|_{L^2}^2 + \|\nabla v\|_{L^2}^2$.
- **Diffusion Tensor:** The tensor $D = d^{\text{ext}} \mathbf{I} + d^{\text{axn}} \mathbf{n} \otimes \mathbf{n}$ is symmetric and positive definite. We assume $d^{\text{ext}}, d^{\text{axn}} \geq 0$.
- **Stability Assumption:** The numerical method is assumed to be stable up to the current state, implying that concentrations remain physical, i.e., $c^n, c_k^{(n+1)} \in [0, 1]$ in Ω .

To apply the Lax-Milgram theorem, we must show that $a(\cdot, \cdot)$ is coercive and continuous on $H^1(\Omega)$.

Coercivity of the Bilinear Form $a(\delta, v)$

We must show that there exists a constant $\alpha_0 > 0$ such that

$$a(v, v) \geq \alpha_0 \|v\|_{H^1(\Omega)}^2 \quad \forall v \in H^1(\Omega).$$

Using the definition of the bilinear form, we have

$$\begin{aligned} a(v, v) &= \int_{\Omega} \frac{1}{\Delta t} v^2 \, d\mathbf{x} + \int_{\Omega} \nabla v \cdot D \nabla v \, d\mathbf{x} - \int_{\Omega} \alpha (1 - 2c_k^{n+1}) v^2 \, d\mathbf{x} \\ &= \int_{\Omega} \left(\frac{1}{\Delta t} - \alpha (1 - 2c_k^{n+1}) \right) v^2 \, d\mathbf{x} + \int_{\Omega} \nabla v \cdot (d^{\text{ext}} \mathbf{I} + d^{\text{axn}} \mathbf{n} \otimes \mathbf{n}) \nabla v \, d\mathbf{x} \\ &= \int_{\Omega} \left(\frac{1}{\Delta t} - \alpha (1 - 2c_k^{n+1}) \right) v^2 \, d\mathbf{x} + \int_{\Omega} d^{\text{ext}} \|\nabla v\|^2 \, d\mathbf{x} + \int_{\Omega} d^{\text{axn}} (\mathbf{n} \cdot \nabla v)^2 \, d\mathbf{x}. \end{aligned}$$

Since $d^{\text{axn}} \geq 0$, the last term is non-negative. Therefore, we can establish the lower bound

$$a(v, v) \geq M \|v\|_{L^2(\Omega)}^2 + N \|\nabla v\|_{L^2(\Omega)}^2,$$

where the constants M and N are defined as the essential infima over the domain:

$$M := \operatorname{ess\,inf}_{\mathbf{x} \in \Omega} \left(\frac{1}{\Delta t} - \alpha (1 - 2c_k^{n+1}) \right), \quad N := \operatorname{ess\,inf}_{\mathbf{x} \in \Omega} d^{\text{ext}}.$$

For coercivity, it is required that $M > 0$ and $N > 0$. In this case, we can set $\alpha_0 = \min(M, N) > 0$, which implies

$$a(v, v) \geq M \|v\|_{L^2(\Omega)}^2 + N \|\nabla v\|_{L^2(\Omega)}^2 \geq \alpha_0 \|v\|_{H^1(\Omega)}^2.$$

This leads to two conditions for well-posedness at each Newton step:

1. $\operatorname{ess\,inf}_{\mathbf{x} \in \Omega} d^{\text{ext}} > 0$, i.e., the diffusion is non-degenerate.
2. $\frac{1}{\Delta t} > \operatorname{ess\,sup}_{\mathbf{x} \in \Omega} \alpha (1 - 2c_k^{n+1}) = \|\alpha(1 - 2c_k^{n+1})\|_{L^\infty(\Omega)}.$

4.3.2 Continuity of the Bilinear Form $a(u, v)$

The bilinear form arises from the left-hand side of the linearized weak formulation and is defined as

$$a(u, v) = \int_{\Omega} \frac{uv}{\Delta t} \, d\mathbf{x} + \int_{\Omega} \nabla u \cdot D \nabla v \, d\mathbf{x} - \int_{\Omega} \alpha (1 - 2c_k^{n+1}) uv \, d\mathbf{x},$$

where the diffusion tensor is

$$D = d^{\text{ext}} \mathbf{I} + d^{\text{axn}} \mathbf{n} \otimes \mathbf{n}.$$

We aim to show that there exists a constant $M > 0$ such that

$$|a(u, v)| \leq M \|u\|_V \|v\|_V \quad \forall u, v \in V,$$

where V is the appropriate Hilbert space (e.g., $H^1(\Omega)$). We analyze the form term by term using the triangle inequality:

$$\begin{aligned} |a(u, v)| &\leq \left| \int_{\Omega} \left(\frac{1}{\Delta t} - \alpha (1 - 2c_k^{n+1}) \right) uv \, d\mathbf{x} \right| \\ &\quad + \left| \int_{\Omega} d^{\text{ext}} \nabla u \cdot \nabla v \, d\mathbf{x} \right| \\ &\quad + \left| \int_{\Omega} d^{\text{axn}} (\mathbf{n} \cdot \nabla u) (\mathbf{n} \cdot \nabla v) \, d\mathbf{x} \right|. \end{aligned}$$

1. Time Derivative and Reaction Term We bound the first term using the Cauchy-Schwarz and Poincaré inequalities:

$$\begin{aligned} \left| \int_{\Omega} \left(\frac{1}{\Delta t} - \alpha(1 - 2c_k^{n+1}) \right) uv \, d\mathbf{x} \right| &\leq \left\| \frac{1}{\Delta t} - \alpha(1 - 2c_k^{n+1}) \right\|_{L^\infty} \int_{\Omega} |uv| \, d\mathbf{x} \\ &\leq \left\| \frac{1}{\Delta t} - \alpha(1 - 2c_k^{n+1}) \right\|_{L^\infty} \|u\|_{L^2} \|v\|_{L^2} \\ &\leq C_P \left(\frac{1}{\Delta t} + \left\| \alpha(1 - 2c_k^{n+1}) \right\|_{L^\infty} \right) \|u\|_V \|v\|_V, \end{aligned}$$

where C_P is the Poincaré constant.

2. Isotropic Diffusion Term The isotropic diffusion term is bounded by Cauchy-Schwarz:

$$\left| \int_{\Omega} d^{\text{ext}} \nabla u \cdot \nabla v \, d\mathbf{x} \right| \leq \|d^{\text{ext}}\|_{L^\infty} \|\nabla u\|_{L^2} \|\nabla v\|_{L^2}.$$

3. Anisotropic Diffusion Term For the anisotropic term, since $\mathbf{n}$ is a unit vector field ($\|\mathbf{n}\|_{L^\infty} = 1$), we have

$$\begin{aligned} \left| \int_{\Omega} d^{\text{axn}} (\mathbf{n} \cdot \nabla u) (\mathbf{n} \cdot \nabla v) \, d\mathbf{x} \right| &\leq \|d^{\text{axn}}\|_{L^\infty} \int_{\Omega} |\mathbf{n} \cdot \nabla u| |\mathbf{n} \cdot \nabla v| \, d\mathbf{x} \\ &\leq \|d^{\text{axn}}\|_{L^\infty} \|\mathbf{n} \cdot \nabla u\|_{L^2} \|\mathbf{n} \cdot \nabla v\|_{L^2} \\ &\leq \|d^{\text{axn}}\|_{L^\infty} \|\nabla u\|_{L^2} \|\nabla v\|_{L^2}. \end{aligned}$$

By combining these bounds, we conclude that the bilinear form $a(u, v)$ is continuous, provided that the concentration c_k^{n+1} is bounded in $L^\infty(\Omega)$ at each iteration, which is physically reasonable since c represents a normalized concentration.

4.3.3 Continuity of the Linear Functional $F(v)$

The linear functional $F(v)$ represents the residual from the right-hand side of the linearized system:

$$F(v) = - \int_{\Omega} \frac{c_k^{n+1} - c^n}{\Delta t} v \, d\mathbf{x} - \int_{\Omega} \nabla c_k^{n+1} \cdot D \nabla v \, d\mathbf{x} + \int_{\Omega} \alpha c_k^{n+1} (1 - c_k^{n+1}) v \, d\mathbf{x},$$

where the diffusion tensor is

$$D = d^{\text{ext}} \mathbf{I} + d^{\text{axn}} \mathbf{n} \otimes \mathbf{n}.$$

We want to show that there exists a constant $\bar{K} > 0$ such that

$$|F(v)| \leq \bar{K} \|v\|_V \quad \forall v \in V,$$

where V is the appropriate Hilbert space (e.g., $H^1(\Omega)$). Using the triangle inequality, we split $F(v)$ into terms without and with gradients of v :

$$|F(v)| \leq \left| \int_{\Omega} \left(\alpha c_k^{n+1} (1 - c_k^{n+1}) - \frac{c_k^{n+1} - c^n}{\Delta t} \right) v \, d\mathbf{x} \right| + \left| \int_{\Omega} \nabla c_k^{n+1} \cdot D \nabla v \, d\mathbf{x} \right|.$$

1. Terms without Gradient of v This term combines the reaction and time-stepping contributions. Using the Cauchy-Schwarz and Poincaré inequalities:

$$\begin{aligned}
 & \left| \int_{\Omega} \left(\alpha c_k^{n+1} (1 - c_k^{n+1}) - \frac{c_k^{n+1} - c^n}{\Delta t} \right) v \, d\mathbf{x} \right| \\
 & \leq \left(\left\| \alpha c_k^{n+1} (1 - c_k^{n+1}) \right\|_{L^\infty} + \left\| \frac{c_k^{n+1} - c^n}{\Delta t} \right\|_{L^\infty} \right) \int_{\Omega} |v| \, d\mathbf{x} \\
 & \leq C \left(\left\| \alpha c_k^{n+1} (1 - c_k^{n+1}) \right\|_{L^\infty} + \left\| \frac{c_k^{n+1} - c^n}{\Delta t} \right\|_{L^\infty} \right) \|v\|_{L^2} \\
 & \leq C' \|v\|_V,
 \end{aligned}$$

where C' depends on the Poincaré constant and the domain volume.

2. Terms with Gradient of v This term arises from the diffusion operator. Splitting into isotropic and anisotropic contributions and applying Cauchy-Schwarz:

$$\begin{aligned}
 & \left| \int_{\Omega} \nabla c_k^{n+1} \cdot D \nabla v \, d\mathbf{x} \right| \\
 & \leq \left| \int_{\Omega} d^{\text{ext}} \nabla c_k^{n+1} \cdot \nabla v \, d\mathbf{x} \right| + \left| \int_{\Omega} d^{\text{axn}} (\mathbf{n} \cdot \nabla c_k^{n+1}) (\mathbf{n} \cdot \nabla v) \, d\mathbf{x} \right| \\
 & \leq (\|d^{\text{ext}}\|_{L^\infty} + \|d^{\text{axn}}\|_{L^\infty}) \|\nabla c_k^{n+1}\|_{L^2} \|\nabla v\|_{L^2}.
 \end{aligned}$$

Conclusion Combining the above bounds, we obtain

$$|F(v)| \leq [C' + (\|d^{\text{ext}}\|_{L^\infty} + \|d^{\text{axn}}\|_{L^\infty}) \|\nabla c_k^{n+1}\|_{L^2}] \|v\|_V.$$

Thus, the linear functional $F(v)$ is continuous, provided the following quantities remain bounded during the iterative process:

- $\|c_k^{n+1} - c^n\|_{L^\infty},$
- $\|\alpha c_k^{n+1} (1 - c_k^{n+1})\|_{L^\infty},$
- $\|\nabla c_k^{n+1}\|_{L^2}.$

These conditions are physically and mathematically reasonable for a well-behaved solution over a finite domain.

Summary of Conditions

The Lax-Milgram theorem guarantees a unique solution for the linear system at each Newton step if the problem data and numerical solution satisfy certain criteria.

Assumptions on Problem Data (assumed to hold):

- The diffusion coefficients and reaction parameter are bounded:
 $\|D_{\text{iso}}\|_{L^\infty}, \|D_{\text{aniso}}\|_{L^\infty}, \|\alpha\|_{L^\infty} < \infty.$
- The isotropic part of the diffusion is non-degenerate:
 $\text{ess inf}_{\Omega} D_{\text{iso}} > 0.$

Assumptions on the Numerical Solution (related to stability):

- Concentrations are bounded: $c^n, c_k^{n+1} \in [0, 1]$.
- The previous iterate is sufficiently regular: $\|c_k^{n+1}\|_{H^1} < \infty$.

Critical Condition for Coercivity and Convergence: Given the assumptions above, the crucial condition that must be satisfied for the problem to be well-posed is the time step restriction:

$$\frac{1}{\Delta t} > \|\alpha \cdot (1 - 2c_k^{n+1})\|_{L^\infty}.$$

Since our stability assumption implies $c_k^{n+1} \in [0, 1]$, we have $|1 - 2c_k^{n+1}| \leq 1$. Therefore, a sufficient condition is $\frac{1}{\Delta t} > \|\alpha\|_{L^\infty}$. This highlights the critical trade-off: a large reaction rate α requires a small time step Δt to ensure the coercivity of the system matrix. If this inequality is violated, the bilinear form may cease to be coercive, leading to the divergence of the Newton iteration and failure of the simulation. To check whether our result holds in practice, we tried to run our code with the following parameters:

- $T = 40$ years
- $\Delta t = 0.5$ years
- $d^{ext} = [6.0, 6.0] \text{ cm}^2/\text{year}$
- $d^{axn} = [3.0, 3.0] \text{ cm}^2/\text{year}$
- $\alpha = [3.0, 3.0] \text{ /year}$

With these parameters we found out that the simulation doesn't converge, since the coercivity condition

$$\frac{1}{\Delta t} > \|\alpha\|_{L^\infty}$$

was violated (here $1/\Delta t = 2 < \|\alpha\|_\infty = 3$). As a result, the bilinear form lost coercivity, the system matrix became indefinite, and the Newton iteration diverged already after the first step.

5 Conclusion

As a first thing, we would like to properly thank with our best regards Professor Johannes Weickenmeier, co-author of the paper used for this work, that kindly provided the original mesh used for the paper.

In this project, we have implemented a numerical model to simulate the prion-like propagation of misfolded proteins, a key mechanism in neurodegenerative diseases. By adopting the Fisher-Kolmogorov equation, we modelled the essential dynamics of protein aggregation and diffusion through a nonlinear reaction-diffusion framework. The methodology—involving a weak formulation, the unconditionally stable Backward Euler method for time discretization, and Newton-Raphson linearization—proved to be a highly effective approach reaching the same results obtained by Professor Weickenmeier in [1].

Our results align with the expected behavior, demonstrating logistic growth coupled with spatial diffusion. This work serves not merely as a proof of concept but as a successful validation of the theoretical model, confirming its power to simulate complex biological phenomena.

While applied here to neurobiology, the finite element framework used in this project is a cornerstone of modern computational science with vast applicability. Reaction-diffusion systems and similar partial differential equations model phenomena across numerous fields. This project, therefore, also serves as a manifest to the versatility of computational methods in translating mathematical models into predictive tools across diverse scientific and industrial domains.

Some ideas for expanding the capabilities of this model:

1. **Develop Patient-Specific Models:** Integrate patient-specific brain geometries obtained from MRI scans. This would transition the model from a general representation to a potential clinical tool for personalized disease progression forecasting.
2. **Solve the Inverse Problem:** Use existing clinical data to work backward and estimate key biological parameters like the local reaction rate or diffusion coefficients. This could help uncover patient-specific disease characteristics.
3. **Improve Computational Efficiency:** Implement adaptive mesh refinement, where the mesh resolution increases only in areas with high activity (i.e., where the concentration gradient is steep), optimizing computational resources and enabling larger, more detailed simulations.

Bibliography

- [1] J. Weickenmeier, M. Jucker, A. Goriely, and E. Kuhl, “A physics-based model explains the prion-like features of neurodegeneration in alzheimer’s disease, parkinson’s disease, and amyotrophic lateral sclerosis,” *Journal of the Mechanics and Physics of Solids*, vol. 124, pp. 264–281, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0022509618307063>